

Appunti di Ingegneria del Software

Appunti basati sulle lezioni di Davide Falessi

A.A. 2025/2026

Indice

1	Teoria della Misurazione e Scale di Misura	2
1.1	Introduzione alla Misurazione del Software	2
1.1.1	Definizioni Formali	2
1.1.2	Proprietà di una Metrica di Qualità	2
1.2	Tassonomia delle Scale di Misura	3
1.2.1	Scala Nominale e Ordinale	3
1.2.2	Scale Numeriche (Intervallo e Rapporto)	3
1.3	Modelli di Qualità: Lo Standard ISO/IEC 25010	4
1.4	Introduzione alle Metriche Object-Oriented	4
2	GQM+S, Stima dei Costi e Planning Poker	5
2.1	GQM+Strategies (Goal-Question-Metric + Strategies)	5
2.1.1	Il Reference Process (I 6 Step del GQM+S)	5
2.2	Introduzione alla Stima dei Costi	6
2.2.1	Componenti del Costo ed Overheads	6
2.2.2	Il Ruolo del Linguaggio di Programmazione	7
2.3	Tecniche di Stima	7
2.4	Planning Poker	7
2.4.1	Dinamiche e Regole del Gioco	8
2.4.2	Vantaggi e Consigli Pratici (Tips)	8
3	Introduzione all'Architettura Software	9
3.1	Cos'è un'Architettura Software	9
3.1.1	Architetture Intended vs Unintended	9
3.2	Il Modello 4+1 di Kruchten	10
3.3	Progettazione e Decision-Making	10
3.3.1	L'importanza dei Requisiti Non Funzionali	10
3.3.2	Trade-offs e il ruolo dell'Architetto	11
3.4	Documentazione: Viste e Punti di Vista	11
3.5	Software Product Lines (SPL)	11
3.6	Stili Architettureali	12
4	Debito Tecnico e Code Smells	14
4.1	Il Debito Tecnico (Technical Debt)	14
4.1.1	Cause e Conseguenze	14
4.1.2	Strategie di Gestione	15
4.2	Introduzione ai Code Smells	15
4.3	Catalogo dei "Worst Smells"	15
4.3.1	Esempi di Worst Smells	16

Capitolo 1

Teoria della Misurazione e Scale di Misura

1.1 Introduzione alla Misurazione del Software

La misurazione nel software engineering non è un fine, ma uno strumento decisionale di supporto. Come ricordano Lord Kelvin e Tom DeMarco, l'assenza di misurazione impedisce il controllo e, di conseguenza, il miglioramento dei processi e dei prodotti[cite: 330, 341].

1.1.1 Definizioni Formali

Secondo i modelli standard, è necessario distinguere tre concetti fondamentali:

- **Misura (Measure):** Rappresentazione numerica di una caratteristica o di un attributo di un prodotto o processo software (es. 5000 LOC, 30 mesi-uomo)[cite: 363, 364, 365, 368].
- **Metrica (Metric):** La scala o il sistema di misurazione utilizzato per quantificare una misura[cite: 352]. Definisce le regole di assegnazione dei valori (es. Complessità Ciclomatica, Densità dei Difetti)[cite: 353, 354].
- **Misurazione (Measurement):** Il processo pratico di assegnazione dei numeri alle misure utilizzando una metrica[cite: 357].

1.1.2 Proprietà di una Metrica di Qualità

Per essere considerata valida, una metrica deve soddisfare i seguenti criteri:

Rilevanza: Deve misurare un attributo significativo e importante del prodotto o processo software[cite: 380].

Oggettività: Il risultato deve basarsi su dati verificabili e riproducibili, non su opinioni o stime soggettive[cite: 381, 382].

Affidabilità e Consistenza: La metrica deve produrre risultati costanti in condizioni diverse e deve essere coerente con gli obiettivi del progetto[cite: 383, 384].

Usabilità: Il calcolo e l'interpretazione devono essere facili, intuitivi e non richiedere sforzi o risorse eccessive[cite: 385, 386].

Nota del Prof

La pertinenza di una metrica è strettamente legata al contesto di business. Un errore critico in un sistema assicurativo potrebbe riguardare l'accuratezza dei calcoli (oracolo) con conseguenze disastrose, mentre per un motore di ricerca come Google la priorità assoluta risiede nella disponibilità del servizio (uptime), dove anche poche ore di downtime globale risultano inaccettabili.

1.2 Tassonomia delle Scale di Misura

La scelta della scala utilizzata dipende dalla natura dei dati misurati e determina quali operazioni statistiche siano matematicamente applicabili[cite: 407, 459, 477].

1.2.1 Scala Nominale e Ordinale

- **Nominale:** Utilizzata per assegnare valori a etichette o categorie senza un ordine intrinseco o magnitudine (es. tipi di difetti: logica, UI, performance, sicurezza)[cite: 417, 418, 419]. Operazioni ammesse: Moda, frequenza[cite: 459].
- **Ordinale:** Prevede un ordine di rango per le categorie, ma la distanza o magnitudine tra i valori non è definita né significativa (es. priorità dei difetti: Critical, High, Medium, Low)[cite: 422, 423, 424]. Operazioni ammesse: Mediana[cite: 459].

Nota del Prof

Le scale ordinali, pur limitando le analisi statistiche alla mediana, offrono spesso una maggiore oggettività rispetto a scale numeriche forzate. Valutare la severità di un bug su una scala 0-100 introduce un'alta soggettività (es. la differenza tra 65 e 70 è arbitraria), mentre una distinzione qualitativa (Blocker vs Non-Blocker) risulta più robusta, netta e oggettiva.

1.2.2 Scale Numeriche (Intervallo e Rapporto)

In queste scale i valori sono assegnati in ordine di rango e la magnitudine della differenza tra i punti ha un significato reale e misurabile[cite: 427, 433].

- **Intervallo:** La magnitudine delle differenze è significativa, ma non possiede uno zero assoluto reale[cite: 427, 428].
- **Rapporto (Ratio):** È il tipo di scala più sofisticato, in cui la magnitudine delle differenze è calcolabile e possiede uno zero assoluto reale (es. LOC, sforzo, tempo)[cite: 433, 434, 435].

Statistiche ammesse per entrambe: Media, Deviazione Standard, Varianza, Range (per la scala di Rapporto sono ammessi anche coefficiente di variazione e media geometrica)[cite: 459].

1.3 Modelli di Qualità: Lo Standard ISO/IEC 25010

Lo standard ISO 25010 definisce una tassonomia dettagliata degli attributi di qualità (qualità del prodotto e qualità in uso)[cite: 33]. Le caratteristiche principali individuate dallo standard sono:

1. **Functional Suitability (Adeguatezza funzionale):** Completezza, correttezza e adeguatezza funzionale[cite: 39, 40, 41, 42].
2. **Performance Efficiency (Efficienza delle prestazioni):** Comportamento temporale, utilizzo delle risorse e capacità strutturale[cite: 43, 44, 45, 46].
3. **Security (Sicurezza):** Riservatezza, integrità, non ripudio, responsabilità e autenticità[cite: 47, 48, 49, 50, 51, 52].
4. **Maintainability (Manutenibilità):** Modularità, riusabilità, analizzabilità, modificabilità e testabilità[cite: 53, 54, 55, 56, 57, 58].

Nota del Prof

È cruciale distinguere sempre il modello di qualità dalle metriche: il modello (come l'ISO 25010) definisce *cosa* significa la qualità fornendone una tassonomia, mentre le metriche definiscono *come* quantificarla operativamente[cite: 30]. L'adozione di questi standard fallisce sistematicamente se manca la "buona fede" aziendale; l'uso delle metriche al solo scopo di ottenere certificazioni formali di facciata, senza mirare a un miglioramento sostanziale del prodotto, svalorizza l'utilità dell'intero sistema di valutazione ingegneristico.

1.4 Introduzione alle Metriche Object-Oriented

Le metriche object-oriented vengono impiegate per misurare la qualità dei sistemi software ad oggetti e identificare tempestivamente difetti di progettazione o futuri problemi di manutenibilità[cite: 68, 69]. Vanno sempre interpretate come indicatori di rischio e valutate in modo comparativo (rispetto a una baseline o a un trend) e non in senso assoluto[cite: 73].

Nota del Prof

In ambito manageriale, è molto più efficace adottare un approccio pratico basato sulle distribuzioni piuttosto che sulle medie globali. Risulta strategicamente più utile concentrare le attività di analisi, refactoring e test mirati esclusivamente sul 10% delle classi che presentano i valori peggiori di complessità (*outliers*), piuttosto che tentare di ottimizzare e riscrivere l'intero sistema in modo uniforme.

Capitolo 2

GQM+S, Stima dei Costi e Planning Poker

2.1 GQM+Strategies (Goal-Question-Metric + Strategies)

L'approccio GQM+Strategies (GQM+S) è una metodologia che combina il metodo classico GQM con l'identificazione delle migliori strategie per raggiungere gli obiettivi. Questo approccio collega a cascata gli obiettivi aziendali alla misurazione:

Business Goals → Software Goals → Questions → Measures → Strategies / Targets.

Nota del Prof

Il CQM originale è stato co-creato dal Professor Cantone. L'estensione "+ Strategies" serve ad associare l'azione tecnica al risultato voluto. Tutto parte dal Business: ogni feature o refactoring viene fatto in primis per raggiungere obiettivi aziendali.

2.1.1 Il Reference Process (I 6 Step del GQM+S)

Il processo si sviluppa in sei step principali per permettere il miglioramento continuo:

0. **Initialize:** Impegno (Commitment), Staffing e Responsabilità.
1. **Characterize:** Definire lo scope dell'applicazione e caratterizzare il contesto ambientale.
2. **Set Goals:** Analisi dello status quo, prioritizzazione degli obiettivi e costruzione della griglia goals-strategies.
3. **Choose Process:** Pianificare l'implementazione delle strategie e organizzare la raccolta/analisi dei dati.
4. **Execute Model:** Esecuzione delle strategie, raccolta dati e feedback.
5. **Analyze Results:** Analisi dei dati, revisione delle strategie e analisi costi/benefici.
6. **Package and Improve:** Adattamento dei goal e registrazione delle "lessons learned".

Nota del Prof

L'ultimo step di "packaging" si basa sul concetto di *Experience Factory*: lo scopo è impacchettare l'esperienza in una knowledge base per riutilizzarla in futuro.

Solo nelle slide (non spiegato a voce)

Esempi di utilizzo tipici:

- **Miglioramento del processo:** Ridurre lead time o rework (Metriche: cycle time, difetti sfuggiti).
- **Project management:** Migliorare prevedibilità (Metriche: milestone burndown, throughput).
- **Supporto decisionale:** Valutare l'adozione di nuove tecnologie (Metriche: esiti progetto pilota).

2.2 Introduzione alla Stima dei Costi

La stima si basa su tre domande fondamentali: quanto sforzo (*effort*) è richiesto, quanto tempo di calendario è necessario e qual è il costo totale dell'attività.

Nota del Prof

Bisogna distinguere l'effort (lavoro puramente impiegato) dal tempo di calendario: un'attività può richiedere poco sforzo ma molto tempo se le risorse non sono subito disponibili.

2.2.1 Componenti del Costo ed Overheads

Il costo include Hardware/Software, viaggi, formazione e l'Effort, che è il fattore dominante (salari e costi sociali). Gli **Overheads** includono:

- Costi degli edifici (riscaldamento, luce).
- Costi di rete e comunicazione.
- Strutture condivise come mensa o biblioteca.

Nota del Prof

Il salario dello sviluppatore a volte non copre neanche il 50% del costo totale per un'azienda a causa degli overhead. Inoltre, il software ha un costo marginale di replicazione quasi nullo: vendere a un secondo cliente non costa nulla di più in termini di produzione.

2.2.2 Il Ruolo del Linguaggio di Programmazione

Solo nelle slide (non spiegato a voce)

Il linguaggio impatta drasticamente su tempi e costi. Ad esempio, l'Assembly richiede molto più tempo per coding (8 settimane per 5000 linee) e testing (10 settimane) rispetto a un linguaggio ad alto livello per lo stesso sistema.

2.3 Tecniche di Stima

IMPORTANTE PER L'ESAME

All'esame non vengono chieste le tecniche a memoria, ma i vantaggi, gli svantaggi e quando usare l'una rispetto all'altra.

- **Algorithmic cost modelling:** Approccio basato su formule e dati storici, generalmente basato sulla dimensione del software (es. COCOMO).
- **Expert judgement:** Uno o più esperti usano la loro esperienza per predire i costi.
 - *Vantaggi:* Metodo relativamente economico; accurato se gli esperti hanno esperienza diretta in sistemi simili.
 - *Svantaggi:* Molto inaccurato se mancano veri esperti nel dominio.
- **Estimation by analogy:** Il costo è calcolato comparando il progetto con uno simile nello stesso dominio.
 - *Vantaggi:* Accurato se sono disponibili dati di progetti passati.
 - *Svantaggi:* Impossibile se non si è mai affrontato un progetto comparabile; richiede un database di costi sistematicamente aggiornato.
- **Parkinson's Law:** Il progetto costa qualunque risorsa sia disponibile. Il lavoro si espande per riempire il tempo disponibile.
 - *Vantaggi:* Non si va mai oltre il budget (*no overspend*).
 - *Svantaggi:* Il sistema rimane solitamente incompleto (*unfinished*).
- **Pricing to win:** Il costo è pari a quanto il cliente può spendere.
 - *Vantaggi:* Permette di vincere il contratto.
 - *Svantaggi:* Bassa probabilità che il cliente ottenga ciò che vuole; i costi non riflettono il lavoro reale richiesto.

2.4 Planning Poker

Il Planning Poker è una tecnica Agile per la stima del consenso, basata sulle carte di James Grenning.

2.4.1 Dinamiche e Regole del Gioco

Ogni partecipante riceve un mazzo di carte con una sequenza numerica, solitamente di **Fibonacci** (1, 2, 3, 5, 8, 13, 21, ecc.).

- **La Sfida:** Il moderatore presenta una *user story* e il team può fare domande al Product Owner.
- **Voto Segreto:** Ogni membro seleziona privatamente una carta che rappresenta la "grandezza" (*size*) del task, considerando tempo, rischio e complessità.
- **Svelamento e Dibattito:** Le carte vengono mostrate insieme. Se non c'è consenso, chi ha dato i valori estremi spiega le proprie ragioni. Il gruppo discute e si vota di nuovo finché non si raggiunge un accordo.

IMPORTANTE PER L'ESAME

Perché i "buchi" di Fibonacci? Servono a forzare la granularità corretta. All'aumentare della grandezza aumenta l'incertezza: la sequenza ci spinge a scegliere tra valori distanti (es. 13 o 21), riflettendo la reale imprecisione della stima per task grandi.

Nota del Prof

In caso di mancato consenso, si sceglie la stima superiore per tutelarsi dalla naturale indole ottimista dell'uomo, che tende a sottostimare i tempi ignorando gli imprevisti.

2.4.2 Vantaggi e Consigli Pratici (Tips)

Il Planning Poker favorisce la collaborazione e fa emergere i problemi tecnici in anticipo attraverso il dibattito.

- **Chi vota:** Solo chi farà materialmente il lavoro. I manager non votano per non influenzare le stime al ribasso.
- **Pareggi:** Se il team è diviso tra due taglie consecutive (es. 5 e 8), si sceglie la maggiore e si procede.

Solo nelle slide (non spiegato a voce)

Suggerimenti per l'efficienza:

- Usare un timer per limitare le discussioni a circa un minuto.
- Includere una carta "I need a break" per segnalare la stanchezza.
- Mantenere una **Baseline** consistente tra le varie iterazioni.

Capitolo 3

Introduzione all'Architettura Software

3.1 Cos'è un'Architettura Software

IMPORTANTE PER L'ESAME

L'architettura software è uno dei pilastri dell'ingegneria del software e rappresenta una delle domande più frequenti in sede d'esame.

Secondo lo standard **IEEE 1471**, l'architettura è definita come:

“L'organizzazione fondamentale di un sistema, incarnata nei suoi componenti, nelle loro relazioni reciproche e con l'ambiente, e i principi che ne governano la progettazione e l'evoluzione.”

L'intento principale, come sostenuto da Kruchten, è fornire un **controllo intellettuale** su sistemi di enorme complessità, permettendo agli stakeholder di comprendere la struttura senza annegare nei dettagli implementativi.

3.1.1 Architetture Intended vs Unintended

Ogni sistema possiede intrinsecamente un'architettura, che può essere classificata in due modi:

- **Intended (Pianificata):** Esplicita e documentata prima della scrittura del codice. Segue l'approccio del *Forward Engineering*.
- **Unintended (Accidentale):** Implicita e spontanea. Emerge quando si scrive codice senza un piano; in questo caso, l'architettura deve essere ricostruita a posteriori (*Reverse Engineering*).

Nota del Prof

Immaginate di costruire una casa: nell'approccio *intended* disegnate prima la pianta. Nell'approccio *unintended* costruite le stanze a casaccio e solo alla fine vi accorgete di che forma ha la casa. Ricordate però il Manifesto Agile: l'architettura ha valore solo se serve a produrre codice migliore, non deve essere burocrazia inutile.

3.2 Il Modello 4+1 di Kruchten

Per gestire la complessità, Kruchten propone di decomporre l'architettura in diverse **viste** basate sul principio della *separation of concerns*.

- **Logical View:** Supporta i requisiti funzionali, ovvero ciò che il sistema deve fare per l'utente finale.
- **Development View:** Si concentra sull'organizzazione dei moduli e dei package nel codice sorgente.

Nota del Prof

Un esempio tipico è lo **stack di Android**, che mostra come il sistema sia stratificato dal Kernel Linux fino alle applicazioni.

- **Dynamic View:** Analizza il comportamento a runtime, i processi e le interazioni tra oggetti. Viene solitamente modellata con i *Sequence Diagram*.

Solo nelle slide (non spiegato a voce)

Completamento del modello:

- **Deployment View:** Si occupa della mappatura fisica del software sui nodi hardware e della distribuzione geografica dei server.
- **Scenarios (il “+1”):** Casi d'uso o sequenze di azioni che fanno da collante tra le altre quattro viste, verificandone la coerenza.

Nota del Prof

Pensate a una casa: potete guardare la pianta elettrica, quella idraulica o quella strutturale. Sono tutte viste diverse dello stesso edificio, necessarie a stakeholder diversi (elettricista, idraulico, ingegnere).

3.3 Progettazione e Decision-Making

Il design architeturale si basa su tre fattori: **Riuso** (di pattern o componenti), **Metodo** (tecniche sistematiche) e **Intuizione** (l'esperienza dell'architetto).

3.3.1 L'importanza dei Requisiti Non Funzionali

Nota del Prof

L'architettura è dettata principalmente dai **Requisiti Non Funzionali** (attributi di qualità come sicurezza, velocità, scalabilità). Questi requisiti sono strutturali: non si può “aggiungere la sicurezza” alla fine del progetto. È come una casa: se volete che sia antisismica, deve nascere così. Non potete renderla antisismica una volta costruita senza abbatterla.

3.3.2 Trade-offs e il ruolo dell'Architetto

L'architetto deve bilanciare forze contrastanti provenienti da stakeholder diversi. Le decisioni sono spesso sub-ottimali e prese "al buio" (quando i requisiti sono ancora vaghi).

Solo nelle slide (non spiegato a voce)

Questo dilemma è rappresentato graficamente dalla scelta tra carne "Cotta" o "Cruda": ogni scelta comporta un vantaggio per uno stakeholder e uno svantaggio per un altro.

3.4 Documentazione: Viste e Punti di Vista

IMPORTANTE PER L'ESAME

Questa è una distinzione fondamentale. Molti studenti confondono i due termini: ricordateli bene per l'esame.

- **Architectural View (Vista):** È la rappresentazione del sistema da una certa prospettiva. È composta da uno o più *modelli* (o diagrammi).
- **Architectural Viewpoint (Punto di Vista):** È l'insieme di regole, convenzioni e linguaggi usati per costruire la vista.

Nota del Prof

Esempio dello stadio: il *Viewpoint* è il concetto teorico di "guardare dalla tribuna stampa" (regole e angolazione). La *View* è la foto specifica che scattate in quello stadio particolare applicando quel punto di vista. In questo corso, Modello e Diagramma sono considerati sinonimi.

3.5 Software Product Lines (SPL)

Una SPL è una famiglia di prodotti che condividono un set comune di funzionalità (**Core Assets**). Si basa sul *Systematic Reuse*.

- **Scoping:** Decidere cosa includere nella piattaforma comune. Se lo scope è troppo largo si sprecono soldi; se è troppo stretto si perde l'opportunità di riuso.
- **Vantaggi:** Riduzione drastica dei tempi e dei costi di sviluppo per i prodotti successivi al primo.

Nota del Prof

Pensate alle auto: la Mercedes usa lo stesso tasto per il finestrino su tutti i modelli. La BMW usa lo stesso motore fisico per vari modelli, limitandone la potenza via software. Nel software il riuso è vitale, ma genera conflitti di budget: il Team A spesso non vuole pagare il costo extra per rendere un modulo riutilizzabile anche dal Team B.

3.6 Stili Architettureali

Lo **Stile Architettureale** è un pattern astratto; l'**Architettura** è la sua applicazione pratica.

1. **Client/Server:** Il server gestisce dati e sicurezza, i client richiedono servizi. Dissaccoppiamento parziale (il server non conosce i client).
2. **Peer-to-Peer:** Ogni nodo è sia client che server.

Nota del Prof

Come BitTorrent: chi scarica deve anche caricare, evitando colli di bottiglia.

3. **Pipes and Filters:** Stream di dati trasformati in sequenza.

Nota del Prof

Usato in sistemi critici come l'avionica. Performance altissime, riuso quasi nullo.

4. **Shared Data:** Comunicazione tramite un database centrale.

Solo nelle slide (non spiegato a voce)

Si divide in **Repository** (l'agente chiede i dati) e **Blackboard** (il sistema notifica gli agenti quando i dati cambiano).

Ecco il codice LaTeX per la **Lezione 4**, strutturato per essere inserito nel tuo file principale tramite . Ho mantenuto lo stile dei box personalizzati e inserito le citazioni basate sui documenti forniti.

Snippet di codice

Capitolo 4

Debito Tecnico e Code Smells

4.1 Il Debito Tecnico (Technical Debt)

Il Debito Tecnico (TD) rappresenta l'applicazione di concetti finanziari al dominio dell'ingegneria del software, aggiungendo criteri di tempo ed economia al classico concetto di manutenibilità.

IMPORTANTE PER L'ESAME

Il debito tecnico si scompone in due concetti finanziari fondamentali che lo distinguono dalla semplice manutenibilità:

- **Capitale (Principal):** Il costo (effort) necessario per eliminare il debito, ovvero il tempo speso per il refactoring.
- **Interessi (Interest):** La penalità futura pagata se il debito non viene eliminato, che si traduce in rallentamento dello sviluppo o maggiore probabilità di bug.

4.1.1 Cause e Conseguenze

Il debito emerge tipicamente per ottimizzazioni a breve termine che creano handicap a lungo termine. Le cause includono la **crescita organica** (aumento naturale della complessità) e **scelte opportunistiche** (trade-off per rispettare le scadenze).

Nota del Prof

Spesso si creano trade-off tra qualità interna e "Time to Market". Seguendo il principio Agile "Working code over comprehensive documentation", gli sviluppatori possono trascurare la modularizzazione o i commenti pur di consegnare software funzionante e farsi pagare .

IMPORTANTE PER L'ESAME

Il debito tecnico impatta sempre la release successiva: se il codice non è manutenibile oggi, lo sviluppo futuro sarà più lento e pronò ai difetti.

Solo nelle slide (non spiegato a voce)

Gli analizzatori statici (come SonarQube) faticano a calcolare gli "Interessi" (il costo futuro), rendendoli attualmente una metrica poco chiara.

4.1.2 Strategie di Gestione

Nota del Prof

Il debito tecnico "scade": se un modulo di scarso valore non verrà mai più modificato, non produrrà interessi. È un errore fare refactoring su codice che non verrà più toccato.

Le strategie di gestione includono:

- **Monitoraggio e Refactoring:** Uso di strumenti come SonarCloud per tracciare il trend delle violazioni.
- **Quality Gate:** Una regola che blocca automaticamente i commit contenenti nuove violazioni.

Nota del Prof

Metafora della barca bucata: la prima cosa da fare non è svuotare l'acqua (vecchio debito), ma tappare il buco (usare il quality gate per non farne entrare di nuovo).

4.2 Introduzione ai Code Smells

Occorre distinguere tra tre concetti:

- **Quality Rule:** Principio ingegneristico validato (es. commenti, bassa complessità).
- **Violation o Code Smell:** Porzione di codice non conforme a una regola di qualità.
- **Defect (Bug):** Problema che impatta la qualità esterna (es. crash).

IMPORTANTE PER L'ESAME

I Code Smells **non sono bug**. Non causano malfunzionamenti immediati ma rendono la manutenzione difficile. Agiscono come un "campo minato", aumentando la probabilità che uno sviluppatore introduca difetti reali.

4.3 Catalogo dei "Worst Smells"

I "Worst Smells" sono violazioni che non portano alcun vantaggio in nessuna circostanza, a differenza di alcuni smell "classici" (come God Class o Code Clones) che possono nascere da trade-off temporali.

4.3.1 Esempi di Worst Smells

- **Modificatori non ordinati:** Crea confusione visiva inutile.
- **Object.finalize() sovrascritto:** In Java, forzare la distruzione manuale è spesso deleterio.
- **Codice commentato:** Sporca la codebase; spesso sono rimasugli di prototipi.
- **Mancanza del "break" in uno Switch:** Causa comportamenti imprevisti proseguendo nei case successivi.
- **Concatenazione di Stringhe con '+' in un ciclo:** Degrada gravemente le performance; il refactoring corretto prevede l'uso di **StringBuilder**.
- **Pessimo Naming:** Esempi includono attributi chiamati A, B o C.

Nota del Prof

Chiamare un metodo `ThisIsAGreatMethod1` non fa risparmiare tempo significativo: non siamo in Formula 1 dove si risparmiano millisecondi.

Solo nelle slide (non spiegato a voce)

L'analisi su 27 progetti Apache ha rivelato che i Worst Smells non sono meno frequenti né più severi degli altri, ma non avendo ragioni di esistere, vanno bloccati sempre tramite Quality Gates.

porcodio